

Skyline • Weather

A fast, accessible, **single-file** weather app. Hourly + 7-day forecasts, air quality, UV

1	0	0	3	7
HTML file	dependencies	build steps	Open-Meteo APIs	day forecast

A fast, accessible, **single-file** weather app. Hourly + 7-day forecasts, air quality, UV index, sun times, weather alerts and a multi-city dashboard for any city in the world.

Quick start

Open `index.html` in any modern browser. That's it.

It works over `file://` (localStorage is permitted in Chrome/Firefox/Edge for local files) and equally well served over HTTP. There is nothing to install and nothing to configure.

```
<!-- Optional: serve it instead of opening the file -->
python -m http.server 8000      # then visit http://localhost:8000
```

HTML

Contents

- [What it does](#)
- [Architecture](#)
- [File map](#)
- [Design system](#)
- [Data sources](#)
- [State & persistence](#)

- [Routing](#)
- [Key subsystems](#)
- [Accessibility](#)
- [Testing](#)
- [Extending the app](#)
- [Known limitations](#)

What it does

FEATURE	WHERE
City search	280 ms debounced autocomplete, 8 results, keyboard + pointer, recent & popular chips
Current conditions	Temperature, condition, feels-like, local-time clock, today's range
Metrics grid	Feels like, humidity, dew point, wind (speed + direction arrow), pressure, visibility, rain, sunshine
Sun & UV	Sunrise/sunset arc with the sun's live position along it, day length, UV index now + daily max on a 0–11+ scale
Air quality	US AQI with colour-coded band, 24 h sparkline, 6 pollutant readouts (PM2.5, PM10, O ₃ , NO ₂ , SO ₂ , CO)
Hourly chart	24 h temperature line + area, precipitation-probability bars, night shading, hover/click/keyboard tooltip, responsive redraw
7-day outlook	Per-day range bars positioned against the week's min/max, precipitation chance, wind
Alerts	Thunderstorms, extreme heat/cold, damaging winds, heavy rain, fog/dangerous visibility, high UV, unhealthy air
Favorites	Star any city; multi-city dashboard sorted warmest / coldest / rainiest / alphabetical
Theming	Light / dark / auto, persisted, respects <code>prefers-color-scheme</code>
Atmosphere	Condition-aware sky gradient, glow, and animated rain / snow / storm effects

Architecture

Three layers, in strict order, inside one `<script>` block:

1. CONFIG constants, DOM refs, pure helpers (parseISO, dur, esc, clamp...)
2. ICONS SVG path data + the WMO-code → label/icon/sky lookup tables
3. APP state → fetch → render, one function per concern
4. EVENTS delegated listeners wired once at the bottom
5. INIT restore state, route, first paint

Rules that keep it maintainable:

Single-file weather app

- **Pure helpers are pure.** `parseISO`, `dur`, `esc`, `num`, `clamp`, `compass`, `flagOf`, `fmtClock`, `wmo`, `skyFor`, `aqiBand`, `uvBand` have no DOM or network dependencies — they are directly unit-testable in Node (see [Testing](#)).
- **Render functions are pure-ish.** Each `renderX()` reads `state` and returns markup or writes into a container. They never fetch.
- **One place fetches.** `getJSON`, `fetchForecast`, `batchForecast`, `fetchAir`, `geocode` — all of them go through `getJSON`, which is the single place that can throw.
- **Requests are sequence-guarded.** `state.req` is a monotonic counter. Every load captures `const seq = ++state.req` and bails out if `state.req !== seq` when it resolves. This prevents a slow dashboard response from overwriting a fast detail response (and vice versa).
- **Optional chaining everywhere.** Open-Meteo omits blocks you didn't request, and `batchForecast` returns an array while `fetchForecast` returns an object — both are read through `?.` and `??`.

File map

<code>index.html</code>	the entire application
<code>README.md</code>	this file

`index.html` is internally divided into 20 banner-commented sections:

#	SECTION	LINES
CSS		
1	Design tokens	12
2	Reset & base	97
3	Atmosphere FX	120
4	Shell (topbar / hero / layout)	149
5	Controls (search, segmented, buttons)	164
6	Cards	219
7	Dashboard	327
8	Alerts / empty & error states / toast	362
9	Weather icons	390
10	Responsive + reduced motion + print	405
JS		
—	Constants & utilities	522
—	Icons (custom line set)	584
—	State	684
—	Theme	731
—	API	748
—	Alerts	801
—	Search combobox	857
—	Routing	923
—	View: dashboard	944
—	View: detail	1079
—	Events	1627
—	Init	1732

Design system

A restrained **Swiss / International Typographic Style** — flat surfaces, hairline rules, one accent colour, generous whitespace, and no decorative effects that don't carry information.

Tokens

All colour and spacing derives from CSS custom properties on `:root`:

TOKEN	LIGHT	DARK	ROLE
<code>--bg</code>	<code>#f1f1ef</code>	<code>#0b0b0c</code>	Page background
<code>--surface</code>	<code>#ffffff</code>	<code>#141416</code>	Cards
<code>--surface-2/3</code>	<code>#f7f7f5</code> / <code>#ecec8</code>	<code>#1a1a1d</code> / <code>#232327</code>	Inset, hover
<code>--line</code> / <code>--line-2</code>	<code>#e3e3de</code> / <code>#cbcbc5</code>	<code>#27272b</code> / <code>#3a3a40</code>	Hairlines
<code>--ink</code> / <code>--ink-2</code> / <code>--ink-3</code>	<code>#111112</code> / <code>#4e4e4a</code> / <code>#7e7e78</code>	<code>#f5f5f4</code> / <code>#aaa9a5</code> / <code>#79797c</code>	Text ramp
<code>--accent</code>	<code>#d12c21</code>	<code>#ff6152</code>	Single accent (buttons, "today")
<code>--ok</code> / <code>--warn</code> / <code>--danger</code>	<code>#1c7a4c</code> / <code>#96620a</code> / <code>#bf2a1b</code>	lightened for dark	Status
<code>--gold</code> / <code>--cool</code> / <code>--warm</code>	<code>#d2951f</code> / <code>#4f8bc9</code> / <code>#d98528</code>	—	Sun, chart, UV markers
<code>--r-sm/r-r-lg/r-pill</code>	<code>8/12/18/999px</code>	—	Radii
<code>--maxw</code>	<code>1200px</code>	—	Content measure

Type: `--font` is Inter with a full system fallback stack; `--mono` for tabular numbers.

Condition-aware sky

Rather than a fixed background, each weather condition resolves a hue/saturation/ lightness triple that feeds a two-stop gradient:

```
:root[data-sky="clear-day"] { --h:207; --s:70%; --l:85%; --d1:15%; --ds:32%; }
:root[data-sky="storm-night"] { --h:241; --s:17%; --l:85%; --d1:9%; --ds:13%; }
```

CSS

`--sky-1` / `--sky-2` are then composed, and a dark-mode block re-maps them (`--ds` / `--d1`, plus a 5% second stop) so night skies stay dark without duplicating every rule. `data-sky` is set from `skyFor(weather_code, is_day)` — 14 values across `clear` | `cloudy` | `fog` | `rain` | `sleet` | `snow` | `storm` × `day` | `night`.

Because it's all variables, a new condition is one CSS rule plus one `skyFor` branch.

Icons

Ten hand-authored SVG line icons (`sun`, `moon`, `partly`, `partly-night`, `cloud`, `fog`, `drizzle`, `rain`, `sleet`, `snow`, `thunder`) on a 24×24 grid, `stroke:currentColor`, `aria-hidden`. They inherit `currentColor` so each condition gets its own tint (`.wi--sun` gold, `.wi--cloud` grey, ...). **No emoji** — they render identically on every OS.

Motion

`prefers-reduced-motion: reduce` disables the atmosphere FX, chart transitions and all animation. A print stylesheet strips the shell and leaves the forecast.

Data sources

All three are free, keyless, CORS-enabled Open-Meteo endpoints.

PURPOSE	ENDPOINT
Geocoding	<code>https://geocoding-api.open-meteo.com/v1/search</code>
Forecast	<code>https://api.open-meteo.com/v1/forecast</code>
Air quality	<code>https://air-quality-api.open-meteo.com/v1/air-quality</code>

Requested variables

```
CURRENT = "temperature_2m,relative_humidity_2m,apparent_temperature,is_day," +  
          "precipitation,weather_code,surface_pressure,wind_speed_10m," +  
          "wind_direction_10m,dew_point_2m"  
  
HOURLY   = "temperature_2m,weather_code,precipitation_probability,is_day,uv_index,visibility"  
  
DAILY    = "weather_code,temperature_2m_max,temperature_2m_min,sunrise,sunset," +  
          "uv_index_max,precipitation_sum,precipitation_probability_max," +  
          "wind_speed_10m_max,sunshine_duration"
```

JS

Always sent: `wind_speed_unit` and `temperature_unit` follow the active unit (`km/h` + `°C` or `mph` + `°F`),
`timezone=auto` so timestamps come back as local wall-clock naive strings plus a `utc_offset_seconds`.

The dashboard requests a **reduced** daily set

(`temperature_2m_max,temperature_2m_min,precipitation_probability_max,wind_speed_10m_max` , `forecast_days=1`)
and a 3-variable current block — it deliberately does not pay for the full detail payload it won't render.

Batching

`batchForecast(places, opts)` sends **one** request with comma-separated coordinates. Open-Meteo returns a JSON array whose order matches the input order, so results are zipped back by index. One network round-trip paints the whole dashboard.

State & persistence

A single plain object, `state`:

Single-file weather app

```
const state = {
  place: null,      // { name, admin, country, cc, lat, lon }
  data: null,       // forecast payload for `place`
  aqi: null,        // air-quality payload (detail only)
  favorites: [],    // places[]
  recents: [],      // place names[]
  unit: "celsius",  // "celsius" | "fahrenheit"
  theme: "auto",    // "auto" | "light" | "dark"
  view: "dash",     // "dash" | "detail"
  req: 0           // request sequence guard
};
```

Persisted to `localStorage` (all writes go through `store.set`, which JSON-encodes and is wrapped in `try/catch` so a full or disabled store degrades instead of throwing):

KEY	VALUE	DEFAULT
<code>skyline.favorites</code>	saved places	<code>[]</code>
<code>skyline.recents</code>	recent search names	<code>[]</code>
<code>skyline.unit</code>	temperature/wind/distance units	<code>"celsius"</code>
<code>skyline.theme</code>	theme preference	<code>"auto"</code>
<code>skyline.last</code>	last-opened detail hash	<code>null</code>

Note: `skyline.last` is **removed** when the app starts at the dashboard, so reopening the file returns you home rather than into a stale city.

Routing

Hash-based, so the app works from `file://` with no server rewrites.

```
#/c/{name}/{admin}/{country}/{lat}/{lon}
```

Each segment is `encodeURIComponent`-ed and literal `/` is re-encoded as `%2F`, so city names containing slashes ("São Paulo / SP") round-trip safely. Coordinates are normalised to 4 decimals. The empty hash is the dashboard.

`parseHash()` is the single parser; `hashFor(p)` the single serialiser. A `hashchange` listener re-enters `openPlace()` / `goHome()`, which makes browser back/forward work naturally — with a duplicate-load guard that skips the reload when the hash change was caused by our own navigation.

Key subsystems

Alerts engine

`buildAlerts(data, aqi)` is a pure function returning `[{ level, title, detail }]`, where `level` is `warn | caution | danger`.

It scans the **next 24 hourly weather codes** for thunderstorms, fog and heavy rain, then checks daily aggregates for temperature extremes, damaging wind and UV, and finally the AQI block. Thresholds are unit-aware (`C ? 40 : 104`), and it is safe to call with a missing `daily` block (`d1 = d.daily || {}`) so the dashboard can reuse it for card badges.

Search combobox

An ARIA-combobox implemented from scratch:

- 280 ms debounce, 8 results, English locale
- `ArrowDown` / `ArrowUp` move `aria-activedescendant`; `Enter` selects; `Escape` closes
- `mousedown` is used (not `click`) so the option is chosen before the input blurs
- a `seq` counter discards out-of-order geocode responses
- `combo.items` is the source of truth; `paintSug()` renders from it

Hourly chart

Hand-rolled SVG, no library:

- 24 h window starting at the current hour, temperature line + gradient area fill
- precipitation-probability bars from the plot baseline, highlighted above 60 %
- night hours shaded as `ch-night` rects derived from the `is_day` series
- hover, click, and **arrow-key** navigation drive a crosshair + tooltip
- axis label density adapts to width (every 6 / 4 / 3 hours)
- a `resize` listener (debounced) and the unit toggle both call `paintChart()`
- the same data is emitted as a visually-hidden `<table>` for screen readers

Geometry lives in `chartGeometry(w)`; `drawChart()` prepares data once, `paintChart()` renders. The split means a resize doesn't re-slice the series.

Sun & UV arc

A quadratic Bézier from sunrise to sunset with the sun marker positioned by

`t = (now - sunrise) / (sunset - sunrise)`. After sunset it switches to **tomorrow's** sunrise from `daily.sunrise[1]` and reports day length, so the countdown stays correct overnight instead of showing an em dash.

Accessibility

Single-file weather app
Semantic landmarks (`header`, `nav`, `main`, `footer`) and one `h1` per view

- Skip link to `#main`
- `role="combobox"` + `aria-expanded` + `aria-controls` + `aria-activedescendant` on search
- `aria-live="polite"` status region announces loaded conditions and errors
- Chart data also present as a screen-reader table; the chart is not the only source
- Visible `:focus-visible` rings; the active suggestion is styled, not just tracked
- `prefers-reduced-motion` and `prefers-color-scheme` both honoured
- Buttons are `<button>`; icon-only controls carry `aria-label`
- Text meets AA contrast in both themes (ink ramp is tuned per theme)

Testing

The app has no test framework — deliberately, since it has no build step. Pure helpers are still testable by extracting the `<script>` body and evaluating it in Node with a DOM stub, which is how the logic below was verified.

```
# 1. Syntax
sed -n '/<script>/,/</script>/p' index.html > skyline.js
node --check skyline.js

# 2. Render (needs Chrome)
chrome --headless=new --disable-gpu --virtual-time-budget=15000 \
  --enable-logging=stderr --log-level=0 --dump-dom \
  "file:///.../index.html#/c/Reykjavik/Capital_Region/Iceland/64.1466/-21.9426"
```

BASH

Coverage of the pure layer: `parseISO` (date-only + naive datetime + garbage), `fmtClock` (midnight / noon / 12-hour rollover), `dur` (0 / minutes / hours / negative / NaN), `flagOf`, `compass`, `esc` (XSS), `num` (null / NaN / rounding), `wmo` and `skyFor` across **all** WMO codes with an assertion that every mapped icon and sky token actually exists, `aqiBand` / `uvBand` boundaries, `buildAlerts` (every alert type, plus null / missing-`daily` / missing-`hourly` inputs), and hash round-tripping.

Gotcha: when scraping the dumped DOM, strip the `<script>` block first — `(?s)<script.*?</script>` — or template literals in the source will match your queries and produce phantom false positives.

Verified in headless Chrome against the live API: detail view for two cities, dashboard with five seeded favorites, the empty state, sun-arc arithmetic, chart element counts (24 bars / 24 dots / 12 night rects), and the dashboard's AQI dots, alert badges and warmest-first sort — with zero console errors throughout.

Extending the app

Add a weather condition

1. Add a row to the `wmo` table: `code: ["Label", "icon-key"]`.

2. Add the icon path to `ICONS`.
3. Add a `skyFor` branch and a `[data-sky="..."]` token block.

Add a metric

1. Add the variable to `CURRENT` (or `HOURLY`).
2. Add a `metricTile(label, ico, val, unit, note, flag)` call in `renderDetail`.

Add a new view

1. Add a `<section>` to the shell with `hidden`.
2. Add the view to `setView()`.
3. Extend `parseHash()` if it needs a URL.

Change thresholds

All alert thresholds live together in `buildAlerts` (~line 803) and are unit-aware. The AQI bands are the `AQI_BANDS` array; the UV scale is `UV_BANDS`.

Known limitations

- **No offline support.** Requires a network connection; there is no service worker or cache. A failed fetch renders an inline retry state.
- **Forecast horizon** is 7 days (Open-Meteo's free tier).
- **AQI is US AQI** (`us_aqi`), not the regional index you'd get from a local agency.
- **Precipitation** is probability-based (`precipitation_probability`) plus a daily total; there is no mm/h intensity forecast.
- **Geocoding bias.** Results are name-ranked by population; obscure places may need the full name or a country suffix.
- **Timezone handling** relies on `timezone=auto`. The app shows the location's local time, not the viewer's.
- Air quality is fetched only for the detail view, not dashboard cards beyond the batch AQI badges — it is the single most failure-prone call, so it is isolated to `Promise.allSettled`.